Module 04 — Strings

Phase: Core Python
Previous: Flow Control
Next: Lists

Module Outcome

Create strings, slice and clean them, split and join pieces, and apply those tools to names, emails, phones, and simple report lines.

By the end of this module you will be able to:

- Write strings with quotes and triple quotes
- Index and slice characters (`start` included, `stop` excluded)
- Treat strings as immutable — methods return a **new** string; rebind to keep it
- Clean input with `strip`, `lower`, `upper`, and `title`
- `split` a line into pieces and `join` pieces back (the pieces come as a list — list methods come next)
- `replace`, mask digits, and check with `in` / `startswith` / `endswith`

---

1. Why This Concept Matters

Numbers are easy to store. Names, emails, phones, and error messages are **text**. A registration form does not arrive tidy:

```
"  ravi kumar  "
"  Admin@College.COM  "
"9876543210"
"2026001,Ravi Kumar,88,76,91"
```

You need to trim spaces, compare emails without caring about case, hide most of a phone on screen, and pull roll / name / marks out of one line.

That work is **string** work. Almost every screen you will build starts as text.

---

2. Create a string

Understand

A string (`str`) is a sequence of characters. `'...'` and `"..."` behave the same. Triple quotes (`"""..."""`) keep line breaks. Python has no separate `char` type — `'A'` is a string of length 1.

How It Works

```
name = "Ananya Sharma"
course = 'Python Full Stack'
bio = """Joined July 2026 batch.
Based in Bengaluru."""
```

```
print(name)
print(bio)
```

Expected Output:

```
Ananya Sharma
Joined July 2026 batch.
Based in Bengaluru.
```

⚠️ WATCH OUT

```
# Wrong — types must match
"42" + 1              # TypeError

# Prefer
int("42") + 1         # 43
```

If a quote appears inside the text, wrap with the other quote style, or escape: `"He said \"ok\""`.

Try It

Print a two-line welcome banner with a triple-quoted string.

---

3. Indexing and slicing

Understand

`s[i]` picks one character. `s[start:stop]` takes a slice — **start included, stop excluded** (same idea as `range`). Indices start at **0**. Negative indices count from the end.

```
P  y  t  h  o  n
  0  1  2  3  4  5   ← positive
 -6 -5 -4 -3 -2 -1   ← negative
```

Use this to take an initial, the last four digits of a phone, or a middle piece — without rewriting the whole string by hand.

How It Works

```
lang = "Python"
print(lang[0])
print(lang[-1])
print(lang[1:4])
```

Expected Output:

```
P
n
yth
```

A phone's last four digits is the same idea:

```
phone = "9876543210"
print(phone[-4:])
```

Expected Output:

```
3210
```

⚠️ WATCH OUT

```
# Wrong — index past the end
"Priya"[10]           # IndexError

# Wrong — thinking stop is included
"Priya"[0:3]          # "Pri"  (3 characters, not through index 3)
```

Check `len(s)` when you are unsure. `s[0:3]` is always three characters if the string is long enough.

Try It

For `name = "Priya"`, print the first character, the last character, and `name[1:4]`.

---

4. Immutability

Understand

Strings cannot change in place. Methods return a **new** string. If you want the name to hold the new value, **rebind**: `city = city.upper()`.

Passing a string around is safe — nothing silently edits the original.

How It Works

```
city = "mumbai"
city.upper()
print(city)

city = city.upper()
print(city)
```

Expected Output:

```
mumbai
MUMBAI
```

`replace` is the same rule — original stays, unless you save the result:

```
status = "pending"
updated = status.replace("pending", "approved")
print(status)
print(updated)
```

Expected Output:

```
pending
approved
```

⚠️ WATCH OUT

```
# Wrong — strings do not support item assignment
s = "paid"
s[0] = "X"            # TypeError

# Prefer — build a new string
s = "X" + s[1:]
```

Calling `.upper()` or `.replace(...)` without rebinding leaves the variable unchanged.

Try It

From `msg = "pending"`, create a new string with `"paid"` via `replace`. Print both to prove the original is unchanged.

---

5. Clean text — `strip`, `lower`, `upper`

Understand

Copied login fields often look like `" PRIYA@COLLEGE.IN "`. Without cleaning, equals-checks fail even when the account exists.

Method	Job
`strip()`	Remove surrounding whitespace
`lower()` / `upper()`	Normalize case
`title()`	Name-style capitalize (handy for display)

Chain them: `raw.strip().lower()`.

How It Works

```
raw = "  hello  "
print(raw.strip())
print(raw.strip().upper())
```

Expected Output:

```
hello
HELLO
```

A login compare after clean-up:

```
stored_email = "admin@company.com"
entered = "  Admin@Company.COM  ".strip().lower()

if entered == stored_email:
    print("Login OK")
else:
    print("Invalid email")
```

Expected Output:

```
Login OK
```

⚠️ WATCH OUT

```
# Wrong — compare the raw field
" Admin@college.in " == "admin@college.in"    # False

# Prefer
entered = raw.strip().lower()
```

Try It

Normalize `" ravi kumar "` with `strip` and `title()`, then print it.

---

6. `split` and `join`

Understand

`s.split(sep)` cuts one string into **pieces**. Those pieces come back as a list — you will learn list methods in the next module. For now, read a piece by index (`parts[0]`) or unpack into names.

`sep.join(pieces)` glues pieces into one string.

Match the **exact** delimiter in the data (`","` vs `", "`).

How It Works

```
items = "Chai,Samosa,Dosa".split(",")
print(items)
print(" | ".join(items))
```

Expected Output:

```
['Chai', 'Samosa', 'Dosa']
Chai | Samosa | Dosa
```

A marks line is the same pattern:

```
record = "2026042|Priya|78|85|72"
roll, name, py, sql, html = record.split("|")
print(f"Roll {roll}: {name} — Python {py}")
```

Expected Output:

```
Roll 2026042: Priya — Python 78
```

⚠️ WATCH OUT

```
# Wrong — delimiter does not match the data
"a,b,c".split(", ")   # ['a,b,c']  — one piece

# Prefer
"a,b,c".split(",")    # ['a', 'b', 'c']
```

Do not call list methods (`append`, `pop`, `sort`) on the split result yet — that is the next module. Indexing and unpacking are enough here.

Try It

Split `"Design UI, Build API, Write tests"` on `", "` and join the pieces with `" | "`. Print the joined line.

---

7. Replace, mask, and checks

Understand

- **`replace`** — substitute text (remember to keep the return value)
- **Mask** — keep some digits, replace the middle with `*`
- **`in`** — substring membership
- **`startswith`** / **`endswith`** — prefix / suffix tests

f-strings (already in I/O) are the everyday way to print a cleaned result.

How It Works

```
phone = "9876543210"
masked = phone[:2] + "*" * 6 + phone[-2:]
print(masked)

filename = "invoice_july.pdf"
print(filename.lower().endswith(".pdf"))
```

Expected Output:

```
98******10
True
```

A lite email check and a readable line:

```
name = "Priya"
emp = "E101"
print(f"Employee {emp}: {name}")

email = "priya@ksr.in"
```

```
if "@" in email and "." in email.split("@")[-1]:
    print("Email looks OK")
```

Expected Output:

```
Employee E101: Priya
Email looks OK
```

⚠️ WATCH OUT

```
# Wrong — ignoring the return value
text.replace("error", "warn")
print(text)           # still the old string

# Prefer
text = text.replace("error", "warn")
```

`email.split("@")[-1]` is the part after `@`. `[-1]` means “last piece” — useful when you only care about the domain.

Try It

Mask `"9876543210"` as `98******10`. Then print `Employee E101: Divya` with an f-string.

Step-by-Step: Build It

One assemble: clean a name, lite-check an email, mask a phone, parse a marks line — the same skills tonight’s kit uses on messy fields.

```
raw_name = "  ravi kumar  "
clean_name = raw_name.strip().title()
print("Name:", clean_name)

email = "ravi@ksr.in"
if "@" in email and "." in email.split("@")[-1]:
    print("Email OK")
else:
    print("Invalid email")

phone = "9876543210"
print("Phone:", phone[:2] + "*" * 6 + phone[-2:])

record = "2026001,Ravi Kumar,88,76,91"
parts = record.split(",")
roll, name = parts[0], parts[1]
m1, m2, m3 = float(parts[2]), float(parts[3]), float(parts[4])
average = (m1 + m2 + m3) / 3
print(f"Roll: {roll} | Name: {name} | Average: {average:.2f}")
```

Expected Output:

```
Name: Ravi Kumar
Email OK
Phone: 98******10
Roll: 2026001 | Name: Ravi Kumar | Average: 85.00
```

Match the delimiter. This line uses `","` — a pipe-separated file would need `"|"`.

Practice Exercises

Attempt each problem before reading the model answer.

1. From `"Python"`, print the slice that yields `"tho"` and the last character.

Hint: Start included, stop excluded.

Model answer:

```
s = "Python"
print(s[2:5])
print(s[-1])
```

Expected Output:

```
tho
n
```

2. Split `"Masala Dosa|80|2"` on `|`. Print `Masala Dosa × 2 = ₹160` using the pieces (cast price and qty).

Model answer:

```
item, price, qty = "Masala Dosa|80|2".split("|")
total = int(price) * int(qty)
print(f"{item} × {qty} = ₹{total}")
```

Expected Output:

```
Masala Dosa × 2 = ₹160
```

3. Normalize login email `" HR@Company.COM "` with `strip` and `lower`, then compare to `"hr@company.com"`. Print `Match` or `No match`.

Model answer:

```
entered = "  HR@Company.COM  ".strip().lower()
if entered == "hr@company.com":
    print("Match")
else:
    print("No match")
```

Expected Output:

```
Match
```

4. Mask `"9123456780"` as `91******80`.

Model answer:

```
phone = "9123456780"
print(phone[:2] + "*" * 6 + phone[-2:])
```

Expected Output:

```
91******80
```

5. Join `["Design", "Code", "Test"]` with `" → "`. (A list literal is fine here — you are feeding `join`, not calling list methods.)

Model answer:

```
print(" → ".join(["Design", "Code", "Test"]))
```

Expected Output:

```
Design → Code → Test
```

6. From `status = "open"`, create `closed` via `replace`. Print both.

Model answer:

```
status = "open"
closed = status.replace("open", "closed")
print(status)
print(closed)
```

Expected Output:

```
open
closed
```

Lab

Combine clean, check, mask, and parse in one short script. No `def`, no files.

Problem

A registration desk pastes one messy name, one email, one phone, and one CSV marks line. You must print a cleaned report without crashing on a bad email shape.

What to Build

A short script that hard-codes the four inputs, cleans them, and prints a small report.

Requirements

- Name: `strip` + `title`
- Email: `strip` + `lower`, then a lite check (`@` present and the domain part contains `.`)
- Phone: 10-digit string → display mask `98******10` style (first 2 + last 2)
- Marks line `roll,name,m1,m2,m3` → print roll, name, and average to 2 decimals
- Print `Email OK` or `Invalid email`

Expected Behavior

Padded / mixed-case name and email become tidy. The original phone digits are not printed in full. Average uses all three marks.

Sample Run

```
Name: Ravi Kumar
Email OK
Phone: 98******10
Roll: 2026001 | Name: Ravi Kumar | Average: 85.00
```

(Your seed values may differ; the **shape** of the output should match.)

Constraints

Top-level statements only — no `def`, no type hints, no files.

Bonus

Reject the email if it still contains a space after `strip`. Print a second mask that shows only the last 4 digits (`******3210`).

Mini-Project (Assignment): Registration Field Cleaner

Why this matters: Forms arrive messy. Email must be comparable, phone digits must be extracted, and screens should not show a full mobile number. That is the same cleaning every signup and ticket form needs.

Lab kit: 15_Lab_Assets/09_string_parser/

File	What you do
`starter/cleaners.py`	Finish the three blocks (email, digits-only phone, mask). **No `def` required.**
`broken/cleaners.py`	Fix trim / country-code / mask slice bugs
`solution/`	Do **not** copy this

This is **not** the Step-by-Step assemble from class. Same string tools, kit-shaped ticket.

Objective

Clean a messy email, keep a 10-digit phone, and mask it for display.

Story / Scenario

Onboarding pastes `" Ravi@Gmail.COM "` and `"+91-98765-43210"`. Staff need a normalized email and a masked number — never a traceback.

Requirements

Piece	Detail
Email	`strip` + `lower`, then print
Phone	Loop characters; keep digits (`ch.isdigit()` or `ch in "0123456789"`); if the result is 12 digits starting with `91`, drop that prefix; keep only a final length of 10
Mask	`******` + last 4 characters of a 10-digit phone
Style	Top-level statements only — no `def`, no type hints, no files

Expected Behavior

- `" Ravi@Gmail.COM "` prints as `ravi@gmail.com`
- `"+91-98765-43210"` becomes `9876543210`
- `"9876543210"` masks as `******3210`

Sample Run

```
ravi@gmail.com
9876543210
******3210
```

(Print order follows the three blocks in the starter.)

Bonus Challenges

- Reject emails that still contain spaces after `strip`
- Mask Aadhaar-style input showing only last 4 digits
- **Later — Files week:** save cleaned rows to a CSV

Done when

- [ ] Email is lowercased and stripped
- [ ] Phone is digits-only and length 10 after dropping a leading `91`
- [ ] Mask shows `******` plus the last 4 digits
- [ ] No `def` / type hints / files on the core path

Interview Q&A

Q1. Are strings mutable? What happens with `s[0] = 'x'`?

No. Strings are immutable. Index assignment raises `TypeError`. Build a new string and rebind.

Q2. `find` vs `index`?
Both search for a substring. `find` returns `-1` if missing; `index` raises `ValueError`.

Q3. What does `s[::-1]` do?
Reverses the string (step `-1`).

Q4. Why prefer `join` over `+` in a loop?
`join` builds once from the pieces; repeated `+` creates many temporary strings.

Q5. Why mask phone numbers on screen?
Privacy — do not dump full numbers into logs or shared displays.

Q6. When is `split` clearer than slicing for parsing?
When the delimiter position varies (email `@`, CSV commas) — split names the pieces; hunting indices does not.

Q7. Why does `name.upper()` leave `name` unchanged unless you rebind?
Methods return a new string; the original object cannot change. Use `name = name.upper()`.

Cheat Sheet

```
"..." / '...'       → create a string
"""..."""           → multi-line string
s[i]                → one character (IndexError if out of range)
s[start:stop]       → slice (stop excluded)
s[-1] / s[-4:]      → last character / last four
len(s)              → length
s.strip()           → trim surrounding whitespace
s.lower() / .upper() / .title()
s.replace(a, b)     → new string (rebind to keep it)
s.split(sep)        → pieces (a list — methods next module)
sep.join(pieces)    → one string
s.startswith / .endswith
"x" in s            → substring check
"*" * 6             → repeat
```

Strings are immutable. Methods return new strings.

Completion Checklist

```
[ ] I can create strings, including a multi-line banner
[ ] I can index and slice (stop excluded; negatives from the end)
[ ] I know strings are immutable and I must rebind
[ ] I can strip / lower / title before comparing
[ ] I can split a line and join pieces
[ ] I can replace, mask digits, and check with in / startswith / endswith
[ ] I completed the lab
[ ] I attempted the field-cleaner mini-project
[ ] I can explain immutability in an interview
```

What's Next?

You can now clean and slice text, and pull fields out of one line.

Next, Lists store **many** values in order — carts, marks, and the pieces that `split` already handed you — and let you add, change, and remove them.